PyTorch Cheat Sheet

Tensors, shapes, and indexing

A1 Create and inspect

```
x = torch.tensor([[1., 2.], [3., 4.]])  # copy data into tensor
x.shape; x.ndim; x.dtype; x.device  # shape / rank / dtype / device
torch.zeros(2, 3); torch.ones(2, 3)
torch.arange(8)                     # [0,8); end excluded
torch.linspace(0, 1, 5)             # 5 points, both ends included
torch.randn(2, 3)                   # normal N(0,1)
torch.randint(0, 27, (32,))         # 32 integers in [0,27)
torch.rand(2, 3)                    # uniform [0,1)
torch.zeros_like(x); torch.randn_like(x)  # same shape / dtype / device
g = torch.Generator().manual_seed(2147483647)  # seeded generator
```

A2 Reshape and axes

```
x.reshape(4, -1)                    # -1 infers size; may copy
x.view(4, -1)                       # no copy; requires compatible strides
x.unsqueeze(0)                      # (T,) -> (1,T)
x.squeeze(1)                        # (B,1,D) -> (B,D)
x.transpose(-2, -1)                 # swap two axes; batched-safe
x.permute(0, 2, 1)                  # specify new axis order based on old axis as input
x.T                                 # 2D transpose; 3D+ reverses ALL axes
x.flatten()                         # flatten to 1D
x.contiguous()                      # contiguous layout; copy if needed
```

A3 Combine and split

```
torch.cat([a, b], dim=0)            # [(2,2),(2,2)] -> (4,2); existing axis
torch.stack([a, b], dim=0)          # [(2,2),(2,2)] -> (2,2,2); turns list of tensor into tensor
a, b = x.split(2, dim=1)            # (4,3) -> ((4,2),(4,1)) - tuple! ; 2 means split size.
q, k, v = qkv.chunk(3, dim=-1)      # split last axis into chunks
```

A4 Math and reductions

```
x + y; x * y; x ** 2; x > 0         # elementwise; broadcasts
x @ W                               # (...,M,K) @ (...,K,N) -> (...,M,N)
x.sum(dim=0)                        # reduce axis 0
x.mean(dim=-1, keepdim=True)  # if x.mean(), reduce to scalar; if keepdim not specified, remove
    # last dim
x.max(dim=1)                        # returns (values, indices)
x.max(dim=1).values                 # values only; also x.amax(dim=1)
x.argmax(dim=-1)                    # indices only
x.item()                            # one-element tensor -> Python number
```

A5 Indexing and masking

```
x[:, 0]; x[:, [0,1]]          # all rows, first column; all rows, first second column
x[[0, 2]]                           # rows 0 and 2
probs[torch.arange(B),[0,1,2]]    # paired row/col indices: [p[0][0], p[1][1], p[2][2]]
x[mask]                             # Boolean selection; copy
emb = Cemb[Xb]                      # (B,T) ids into (V,D) -> (B,T,D)
torch.where(cond, a, b)             # elementwise choose a or b
torch.tril(a)                       # fills top right triangle with 0s; causal mask
x.masked_fill(m == 0, float('-inf'))  # replace wherever mask is true
```

A6 Broadcasting

```
# Right-align shapes. Equal -> keep; either is 1 -> stretch; otherwise ERROR.
# Missing left axes count as 1. Shape alignment does not zero-pad the data.
# (3,3)+(3,1) -> (3,3); (3,3)+(3,2) -> ERROR.
x = tok_emb + pos_emb               # nanoGPT: (B,T,C) + (T,C) -> (B,T,C)
A = torch.randn(4, 3, 2); B_ = torch.randn(2, 3)
(A @ B_).shape                      # B_ -> (1,2,3) -> (4,2,3); result: (4, 3, 3)
# Matmul: last two axes multiply; all leading axes broadcast.
```

PyTorch Cheat Sheet

Autograd, modules, and data

A7 Autograd

```
# Manual SGD update is included below the autograd calls.
w = torch.randn(10, requires_grad=True)  # leaf; stores gradients in w.grad
loss = (w ** 2).sum()              # forward builds the graph
loss.backward()                    # ACCUMULATES gradients
w.grad                             # same shape as w
w.detach()                         # shared data; gradient path cut
with torch.no_grad(): ...          # do not record gradients
with torch.inference_mode(): ...   # inference without graph recording
for p in parameters:
    p.data += -lr * p.grad
```

A8 Modules and layers

```
# Lowercase: do it. Capitalized: construct the thing that does it.
class MLP(nn.Module):
    def __init__(self, n_in, n_hidden, n_out):
        super().__init__()         # initialize parent before modules
        self.net = nn.Sequential(
            nn.Linear(n_in, n_hidden),  # weight: (out,in)
            nn.ReLU(),
            nn.Linear(n_hidden, n_out),
        )
    def forward(self, x):
        return self.net(x)         # call model(x)
nn.Embedding(vocab_size, n_embd)   # (B,T) ids -> (B,T,D)
nn.LayerNorm(n_embd); nn.Dropout(0.2)
nn.ModuleList([Head(hs) for _ in range(n_head)])  # registers contained parameters, BUT NOT
    # directly callable. Need a for loop to call
self.register_buffer('tril', torch.tril(torch.ones(T, T)))
F.relu(x); F.softmax(x, dim=-1)    # functional layer calls
model.parameters()                 # parameters for the optimizer
probs = torch.softmax(logits, dim=-1)  # function: calculate now
probs = logits.softmax(dim=-1)     # tensor method: same calculation
softmax_layer = nn.Softmax(dim=-1)  # class: construct a layer
probs = softmax_layer(logits)      # call the layer
# Buffers move/save with the model; no optimizer update. Softmax has no parameters.
```

A9 Losses

```
# Cross-entropy includes log-softmax; do not apply softmax first.
F.cross_entropy(logits, y)         # RAW logits (B,C); integer y (B,)
loss = F.cross_entropy(logits.view(B*T, C), targets.view(B*T))
F.cross_entropy(logits, y, ignore_index=-100)  # ignore padded labels
F.binary_cross_entropy_with_logits(logits, y_float)  # binary / multi-label; raw logits
```

A10 Data pipeline

```
class MyDataset(Dataset):          # Dataset template
    def __init__(self, ...): ...  # initialize once
    def __len__(self): return N    # number of examples
    def __getitem__(self, i): return sample  # return one example
def collate(samples):              # combine a batch; e.g. ragged input
    xs = [s['x'] for s in samples]
    return pad_sequence(xs, batch_first=True, padding_value=-1)  # pad to rectangle
loader = DataLoader(dataset, batch_size=512, shuffle=True,
                    collate_fn=collate, num_workers=4, pin_memory=True)
for batch in loader:               # indices -> samples -> collate
    ...
```

PyTorch Cheat Sheet

Training, sampling, and GPT examples

A11 Train eval save load freeze

```
model.train()
for xb, yb in loader:
    xb, yb = xb.to(device), yb.to(device)
    optimizer.zero_grad(set_to_none=True)  # clear accumulated gradients
    logits = model(xb)
    loss = F.cross_entropy(logits, yb)
    loss.backward()                # ACCUMULATES gradients
    optimizer.step()
model.eval()                       # Dropout / BatchNorm eval behavior
with torch.inference_mode():       # disable graph recording separately
    preds = model(x).argmax(dim=-1)
torch.save(model.state_dict(), 'ckpt.pt')
model.load_state_dict(torch.load('ckpt.pt'))  # strict=False permits missing keys
for p in model.backbone.parameters():
    p.requires_grad = False        # freeze parameter
```

A12 Sampling loop

```
for _ in range(max_new_tokens):
    logits, _ = model(idx[:, -block_size:])  # crop to context window
    probs = F.softmax(logits[:, -1, :], dim=-1)  # last position only -> (B, V)
    idx_next = torch.multinomial(probs, num_samples=1)  # sample one id per row
    idx = torch.cat((idx, idx_next), dim=1)  # append, repeat
```

A1 GPT and makemore examples

```
# makemore p3
Cemb = torch.randn((vocab_size, n_embd), generator=g)  # embedding table (V, D)
ix = torch.randint(0, Xtr.shape[0], (batch_size,), generator=g)  # random row indices
```

A2 GPT and makemore examples

```
# makemore p3/p4
emb = Cemb[Xb]                     # (B,T) ids into (V,D) -> (B,T,D)
embcat = emb.view(emb.shape[0], -1)  # (B, 30)     the -1 computes 3*10
# nanoGPT
k = k.view(B, T, n_head, head_size).transpose(1, 2)  # (B, T, C) -> (B, nh, T, hs)
y = y.transpose(1, 2).contiguous().view(B, T, C)  # (B, nh, T, hs) -> (B, T, C)
# makemore p5
x = x.view(B, T // n, C * n)       # (B, 8, 10) -> (B, 4, 20)
# makemore p5
convolution = conv(x.transpose(1, 2)).transpose(1, 2)
# images
x.permute(0, 2, 3, 1)              # (B, C, H, W) -> (B, H, W, C)
```

A3 GPT and makemore examples

```
# gpt_dev get_batch
x = torch.stack([data[i:i+block_size] for i in ix])  # list of (T,) -> (B, T)
y = torch.stack([data[i+1:i+block_size+1] for i in ix])  # targets, shifted by one
# gpt_dev generate
idx = torch.cat((idx, idx_next), dim=1)  # (B, T) -> (B, T+1)
# nanoGPT
q, k, v = self.c_attn(x).split(n_embd, dim=2)  # (B,T,3C) -> 3 x (B,T,C)
# gpt_dev multi-head
out = torch.cat([h(x) for h in self.heads], dim=-1)  # nh x (B,T,hs) -> (B,T,C)
```

A4 GPT and makemore examples

```
# makemore p4
bnmeani = hprebn.mean(0, keepdim=True)  # (1, H)  per-neuron mean over batch
logit_maxes = logits.max(1, keepdim=True).values  # (B, 1)  per-row max
norm_logits = logits - logit_maxes  # (B, C)  broadcasts back over rows
```

A5 GPT and makemore examples

```
# makemore p4
loss = -logprobs[range(n), Yb].mean()  # == F.cross_entropy(logits, Yb)
# gpt_dev
wei = wei.masked_fill(self.tril[:T, :T] == 0, float('-inf'))
wei = F.softmax(wei, dim=-1)       # -inf -> exactly 0 attention
```